PROJECT APPEND EXCEL FILES FROM GMAIL

A step-by-step workflow using Gmail, Google Apps Script, a Web App URL and Microsoft Excel Power Query

1. Gmail	2. Apps Script	3. Web App URL	4. Power Query	5. Transform	6. Consolidate
Excel attachments arrive	Read email + encode file	Return JSON response	Connect From Web	Decode, extract, expand	Append + load

PROJECT PURPOSE

Retrieve Excel attachments delivered through Gmail, expose their email metadata and file content through a Google Apps Script Web App, then use Power Query to convert the encoded content into binary, read the workbook tables, expand the required data, and prepare a consolidated dataset.

WORKFLOW EXPLAINED

STEP 1 — Prepare Gmail

Open Gmail Settings → See all settings → Forwarding and POP/IMAP. Your notes record enabling POP and IMAP and saving changes. This is the setup you documented; the GmailApp script itself accesses Gmail through Apps Script permissions and does not normally require POP/IMAP.

STEP 2 — Create the Apps Script

Visit script.google.com/home → New project. Paste the script, save it, and use the project to retrieve email message details and attachment bytes. The supplied script checks the first message in each of the latest 10 inbox threads and returns attachment name, MIME type, date, sender, subject and Base64 content.

STEP 3 — Deploy as a Web App

Choose Deploy → New deployment → Web app, authorize the requested permissions, deploy, and copy the Web App URL. Your notes specify access as “Anyone”; use the least-permissive setting that works for your use case and never share a URL that exposes private email data.

STEP 4 — Connect Power Query to the URL

In Excel choose Data → Get Data → From Other Sources → From Web. Paste the Web App URL and confirm. Power Query reads the JSON response and converts it into a table so each email/attachment record can be inspected.

STEP 5 — Filter Excel attachments

Filter the FileName column to keep values ending in .xlsx (or use a case-insensitive text filter). This keeps the workflow focused on Excel files even though the supplied script returns all attachment types.

STEP 6 — Decode and read the workbook

Add a Custom Column using Binary.FromText([Binary], BinaryEncoding.Base64). Then add another Custom Column using Excel.Workbook([Custom], false, true), replacing [Custom] with the name of the decoded binary column.

POWER QUERY TRANSFORMATION & SCRIPT

Technical reference for the project: decode the attachment, extract worksheet data, standardize the table and append records

A. Base64 text → Binary `Binary.FromText([Binary], BinaryEncoding.Base64)`	**B. Binary → Workbook tables** `Excel.Workbook([ExcelBinary], false, true)`
Creates a real binary value from the Base64 text. Name this new column ExcelBinary.	Reads workbook structure and returns sheet/table metadata plus nested Data tables. Replace [ExcelBinary] with your decoded column name.

GOOGLE APPS SCRIPT — SOURCE CODE

```
function doGet() {
  var threads = GmailApp.getInboxThreads(0, 10);
  var emailData = [];

  threads.forEach(function(thread) {
    var msg = thread.getMessages()[0];
    var attachments = msg.getAttachments();

    attachments.forEach(function(att) {
      emailData.push({
        EmailDate: msg.getDate().toISOString(),
        From: msg.getFrom(),
        Subject: msg.getSubject(),
        FileName: att.getName(),
        MimeType: att.getContentType(),
        Binary: Utilities.base64Encode(att.getBytes())
      });
    });

    if (attachments.length === 0) {
      emailData.push({
        EmailDate: msg.getDate().toISOString(),
        From: msg.getFrom(),
        Subject: msg.getSubject(),
        FileName: "No Attachment",
        MimeType: "N/A",
        Binary: "N/A"
      });
    }
  });

  return ContentService.createTextOutput(JSON.stringify(emailData))
    .setMimeType(ContentService.MimeType.JSON);
}
```

HOW THE FINAL APPEND WORKS

Source file A	Source file B	Appended result
OrderID | Product | Sales	OrderID | Product | Sales	OrderID | Product | Sales
1001 | Laptop | 150000	1003 | Keyboard | 12000	1001 | Laptop | 150000
1002 | Mouse | 7500	1004 | Monitor | 45000	1002 | Mouse | 7500
		1003 | Keyboard | 12000
		1004 | Monitor | 45000

Technical accuracy notes: The uploaded script returns all attachment types; the .xlsx-only selection is applied in Power Query by filtering FileName. It examines the first message in each of the latest 10 inbox threads, not necessarily the newest message in each thread. The script output does not itself append worksheet rows: Power Query must extract and standardize each workbook table, then append compatible tables. The Web App URL can expose email metadata and attachments depending on deployment permissions—keep it private.

LinkedIn-ready project summary: Built a Gmail-to-Power Query workflow using Google Apps Script and a Web App endpoint to retrieve email attachment metadata, decode Base64 file content, extract Excel workbook tables, and prepare consistent datasets for append and consolidation. Key skills: Google Apps Script, JSON, Power Query M, binary conversion, Excel.Workbook, data cleaning and append workflows.